

C:\Users\mthob\OneDrive\Documents\NetBeansProjects\FederatedIDSArtefact\src\main\java\com\registration\app\federatedid

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  */
4  package com.registration.app.federatedidsartefact;
5
6  /**
7   * 29/01/2026
8   * @author mthobi
9   */
10 import java.util.List;
11 import java.util.ArrayList;
12 import java.util.stream.Collectors;
13
14 public class FederatedIDSArtefact
15 {
16
17     public static void main(String[] args)
18     {
19         String path = "C:/Users/mthob/OneDrive/Documents/NetBeansProjects/FederatedIDSArtefact/data/iot_da
20         System.out.println("Reached main() start");
21         System.out.println("JVM working directory: " + System.getProperty("user.dir"));
22         //System.out.println("");
23         System.out.println("Starting Federated IDS Artefact...\n");
24         System.out.println("Dataset exists? " + new java.io.File(path).exists());
25 // =====
26         // 1. LOAD DATASET (THIS IS WHERE YOUR CODE GOES)
27         // =====
28         //String path = "data/iot_dataset.csv";
29         //String path = "C:/Users/mthob/OneDrive/Documents/NetBeansProjects/FederatedIDSArtefact/data/iot_
30
31         List<double[]> features;
32         List<Integer> labels;
33
34         try
35         {
36             features = DataLoader.loadFeatures(path);
37             labels = DataLoader.loadLabels(path);
38         } catch (Exception e)
39         {
40             System.err.println("✗ Failed to load dataset. Check file path: " + path);
41             e.printStackTrace();
42             return; // STOP execution safely
43         }
44
45         // =====
46         // 2. CREATE FEDERATED CLIENTS
47         // =====
48         int numClients = 3;
49         int total = features.size();
50         int splitSize = total / numClients;
51
52         List<Client> clients = new ArrayList<>();
53
54         for (int i = 0; i < numClients; i++)
55         {
56
57             double[][] X = new double[splitSize][];
58             int[] y = new int[splitSize];
59
60             for (int j = 0; j < splitSize; j++)
61             {
62                 X[j] = features.get(i * splitSize + j);
63                 y[j] = labels.get(i * splitSize + j);
64             }
65
66             boolean malicious = (i == 0); // first client malicious
67             clients.add(new Client(X, y, malicious));
68
69             System.out.println(
70                 "Client " + (i + 1)
```

```

71         + (malicious ? " (MALICIOUS)" : " (BENIGN)")
72         + " samples: " + splitSize
73     );
74 }
75
76
77
78
79
80
81 // =====
82 // 3. DRIFT DETECTOR
83 // =====
84 DriftDetector driftDetector = new DriftDetector(0.95); // high threshold to force demo
85
86 // =====
87 // 4. FEDERATED TRAINING ROUNDS
88 // =====
89 for (int round = 1; round <= 3; round++)
90 {
91     List<Model> localModels = new ArrayList<>();
92
93     for (Client c : clients)
94     {
95         localModels.add(c.trainLocalModel());
96     }
97
98     Model globalModel = FederatedServer.federatedAverage(localModels);
99
100     double acc = Metrics.accuracy(
101         globalModel,
102         features.toArray(new double[0][]),
103         labels.stream().mapToInt(i -> i).toArray()
104     );
105
106     System.out.println("Round " + round + " accuracy: " + acc);
107
108     if (driftDetector.detectDrift(acc))
109     {
110         System.out.println("⚠ Concept drift detected! Triggering adaptation...");
111
112         for (Client c : clients)
113         {
114             c.localModel = new Model(c.X[0].length);
115         }
116
117         System.out.println("✅ Model adaptation completed.");
118     }
119 }
120
121 System.out.println("\nFederated IDS Artefact Execution Completed.");
122 }
123 }
124 }
125

```